

AOS1 A2026

TP 3 — Bayesian linear regression

numpy=1.26.4; seaborn=0.13.2; matplotlib=3.10.1; pandas=2.2.0

1 Introduction

This practical session aims at applying Gaussian process regression. You will need several libraries for this purpose.

```
import numpy as np
import scipy as sp
import scipy.stats as spst
import matplotlib.pyplot as plt
```

2 Practice

2.1 Programming Gaussian process regression

[1] Take a function such as e.g. $f(x) = x \cos(x)$, with $x \in \mathbb{R}$. Generate training, prediction and plot data accordingly, without noise.

[2] Create a function for computing covariances as per the squared exponential kernel in the following formulation:

$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{1}{2\ell^2} \|\mathbf{x} - \mathbf{x}'\|^2\right).$$

You can take default values $\sigma_f^2 = 1$ and $\ell = 1$. You can use the fact that for two design matrices X_1 and X_2 of respective dimensions $n_1 \times 1$ and $n_2 \times 1$, the matrix of pairwise Euclidean distances between the inputs $\mathbf{x} \in X_1$ and $\mathbf{x}' \in X_2$ can be computed as

$$D(X_1, X_2) = \mathbf{s}_1 \mathbf{s}_2^\top - 2X_1 X_2^\top,$$

where $\mathbf{s}_1$ and $\mathbf{s}_2$ respectively contain the square norms of the inputs $\mathbf{x} \in X_1$ and $\mathbf{x}' \in X_2$, i.e., we have $s_{1,i} = \sum_j x_{ij}^2$ and $s_{2,i} = \sum_j x'_{ij}{}^2$.

```
def SE_kernel(X1, X2, l=1, sigma_f=1):
    ...
```

[3] Program a function which computes the predictive distribution parameters for a squared exponential kernel, given test data $\mathbf{X}_{\text{pr}}$, training data $\mathbf{X}_{\text{tr}}$ and $\mathbf{y}_{\text{tr}}$, a noise variance σ_n^2 , a length-scale ℓ , and a signal variance σ_f^2 .

You may use the `np.linalg.inv` function for this purpose. You can also try to optimize the function using Cholesky decomposition.

Reminder 1. Assume you want to solve the system $A\mathbf{x} = \mathbf{b}$ for $\mathbf{x}$ with A a symmetric positive definite matrix. You can proceed as follows:

1. factorize A using Cholesky decomposition, i.e. determine the lower triangular matrix L such that $A = LL^\top$;
2. solve $L\mathbf{y} = \mathbf{b}$ for $\mathbf{y}$, and then solve $L^\top \mathbf{x} = \hat{\mathbf{y}}$ for $\mathbf{x}$ (with $\hat{\mathbf{y}}$ the solution previously obtained).

```
def pred_dist(Xstar, X, y, sigma_n=1e-10, l=1, sigma_f=1):
    if len(Xstar.shape)==1:
        Xstar = Xstar.reshape(-1,1)

    ...
```

[4] Display the predictive distribution obtained. Represent the credibility intervals obtained using the following code. You can also display several functions sampled from the predictive distribution.

```
Xpl = np.linspace(0,10,100)
Xpl = Xpl.reshape(-1,1)

mean_GPR, CovM_GPR = pred_dist(Xpl, Xtr, ytr, sigma_n=sigma_n, l=1, sigma_f=1)

... 
```

2.2 Scikit-learn implementation

[5] We first consider the noise-free case. Build the GPR model; fit the model to the training data, make predictions, and display.

[6] We now study the effect of noise in the model outputs.

```
ytr_noi = model(Xtr.ravel(), 1)
```

[6 a] Train the model from noisy data *assuming they are noise-free*, and display the results.

[6 b] Display the outputs of a model which assumes the presence of noise in the training data, with a fixed amount of noise (using the GPR parameter `alpha`, set for instance to `alpha=1`).

3 Theory

3.1 GPR, weight space view

[7] Show that the “straightforward formulation” of the GPR model, i.e.

$$\mathbf{f}^* \mid \Phi^*, \Phi, \mathbf{y} \sim \mathcal{N}(\Phi^* \bar{\mathbf{w}}, \Phi^* A^{-1} \Phi^{*\top}), \quad \bar{\mathbf{w}} = \sigma_n^{-2} A^{-1} \Phi^\top \mathbf{y}, \quad A = \sigma_n^{-2} \Phi^\top \Phi + \Sigma_p^{-1},$$

can be re-expressed as

$$\mathbf{f}^* \mid \Phi^*, \Phi, \mathbf{y} \sim \mathcal{N}\left(\Phi^* \Sigma_p \Phi^\top (K + \sigma_n^2 I_n)^{-1} \mathbf{y}, \Phi^* \Sigma_p \Phi^{*\top} - \Phi^* \Sigma_p \Phi^\top (K + \sigma_n^2 I_n)^{-1} \Phi \Sigma_p \Phi^{*\top}\right),$$

with $K = \Phi \Sigma_p \Phi^\top$.

3.2 GPR, function space view

[8] Assume we have a training set of instances stored in the $n \times p$ matrix X associated with noisy function outputs stored in the $n \times 1$ vector $\mathbf{y}$. Assume we also have a set of test (prediction) instances stored into the $m \times p$ matrix X^* . We recall that the joint distribution of $\mathbf{y}$ and $\mathbf{f}^*$ writes as

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}^* \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}, \begin{bmatrix} K(X, X) + \Sigma_n & K(X, X^*) \\ K(X^*, X) & K(X^*, X^*) \end{bmatrix} \right).$$

Show that in this case, the predictive distribution of the model outputs for the test instances writes as

$$\mathbf{f}^* | X^*, X, \mathbf{y} \sim \mathcal{N} \left(\overline{\mathbf{f}^*}, \text{Cov}(\mathbf{f}^*) \right),$$

with

$$\begin{aligned} \overline{\mathbf{f}^*} &= K(X^*, X) (K(X, X) + \Sigma_n)^{-1} \mathbf{y}, \\ \text{Cov}(\mathbf{f}^*) &= K(X^*, X^*) - K(X^*, X) (K(X, X) + \Sigma_n)^{-1} K(X, X^*). \end{aligned}$$

You can use the following property of the multivariate Gaussian distribution.

Property 1. Let $\mathbf{X}_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \Sigma_{11})$ and $\mathbf{X}_2 \sim \mathcal{N}(\boldsymbol{\mu}_2, \Sigma_{22})$ be two multivariate Gaussian random vectors with joint distribution

$$\begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}, \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \right);$$

then, the conditional distribution of $\mathbf{X}_1 | \mathbf{X}_2 = \mathbf{x}_2$ is

$$\mathbf{X}_1 | \mathbf{X}_2 = \mathbf{x}_2 \sim \mathcal{N} \left(\boldsymbol{\mu}_1 + \Sigma_{12} \Sigma_{22}^{-1} (\mathbf{x}_2 - \boldsymbol{\mu}_2), \Sigma_{11} - \Sigma_{12} \Sigma_{22}^{-1} \Sigma_{21} \right).$$

3.3 Marginal likelihood

[9] Show that the marginal likelihood writes as

$$\log p_{\mathbf{Y}}(\mathbf{y} | X) = -\frac{n}{2} \log(2\pi) - \frac{1}{2} \log |K(X, X) + \Sigma_n| - \frac{1}{2} \mathbf{y}^\top (K(X, X) + \Sigma_n)^{-1} \mathbf{y}.$$

You can use the property that the product of two multivariate Gaussians is a multivariate Gaussian:

$$p_{\mathcal{N}}(\mathbf{x} | \mathbf{a}, A) p_{\mathcal{N}}(\mathbf{x} | \mathbf{b}, B) = Z^{-1} p_{\mathcal{N}}(\mathbf{x} | \mathbf{c}, C), \quad \mathbf{c} = C (A^{-1} \mathbf{a} + B^{-1} \mathbf{b}), \quad C = (A^{-1} + B^{-1})^{-1}.$$